Intelligent Water Intake Reminder System

Arduino Water Reminder

Authors

Shah Zaman Haider
Syed Taqi Shah

Submitted To

Hamza Chaudhary

6 January 2026

Contents

1 Introduction **1**

2 Simulation on Tinkercad **1**

3 Overview of a Project **2**

4 Arduino UNO **3**

5 Other Hardware Components and Their Purpose **4**

6 Pin Configuration **6**

7 Software Design Methodology **6**

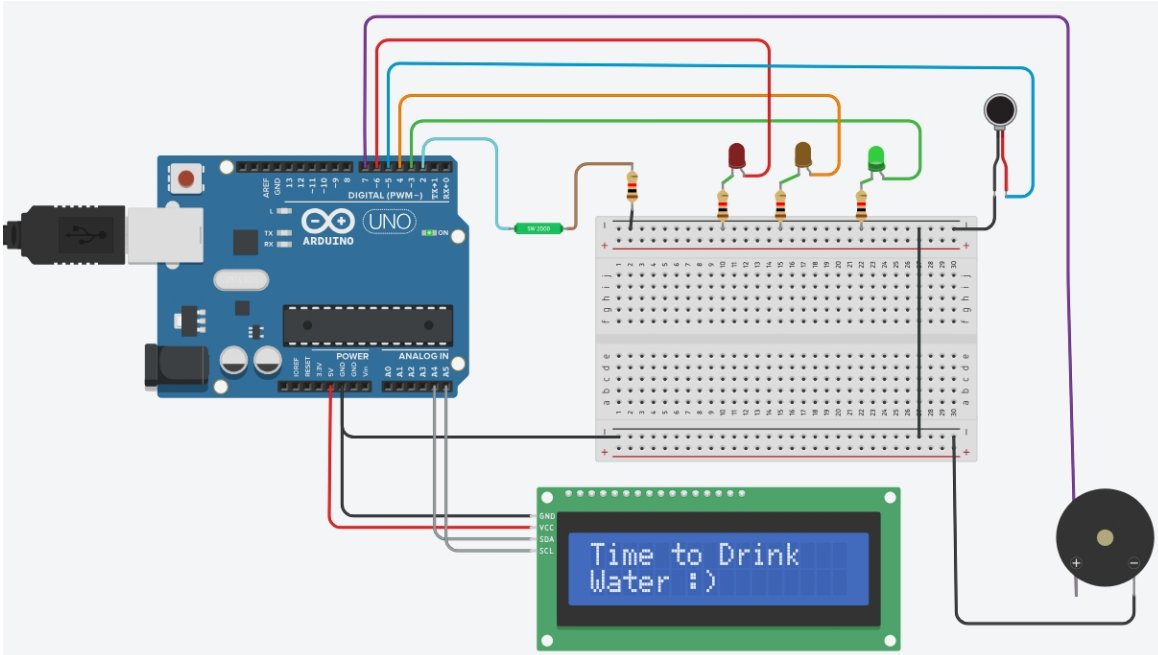
8 System States **6**

9 LCD Display Operation **7**

10 Limitations **7**

11 Future Scope **8**

12 Conclusion **8**



1 Introduction

We all know staying hydrated is crucial for health and focus, yet it is all too easy to forget to drink water when we are deep in the "zone" of studying or working, leading to unnecessary fatigue and headaches. To address this common issue, we designed the Smart Water Bottle Reminder System, a smart attachment that intelligently monitors your drinking habits and reminds you if you've been inactive for too long to prevent dehydration. Instead of a random annoying alarm, its a system that runs from a gentle visual to physical vibrations and finally sound, ensuring you get the message through multiple senses while demonstrating how simple hardware can solve real-world well-being problems.

2 Simulation on Tinkercad

For building, prototyping, and testing our project, we relied heavily on Tinkercad. If you aren't familiar with it, it is an awesome, free online tool by Autodesk that lets you simulate electronics and Arduino code on your computer screen before you ever touch a real wire.

We chose to use Tinkercad for a few critical reasons that made our lives much easier:

- **Safety First:** We didn't want to risk burning out any components, like frying an LED or shorting the Arduino, while we were still figuring out the basics. The simulation lets you make mistakes without any cost.

- **Easy Fixes and Iteration:** If we made a mistake in the wiring or logic, we could fix it in seconds by dragging a wire or rewriting a line of code, rather than having to physically unplug messy wires and hunt for loose connections on a breadboard.
- **Logic Testing:** It allowed us to verify that our code logic was working perfectly. We could speed up the simulation time to test how the bottle behaves after "an hour" without actually waiting that long in real life.

Basically, using Tinkercad saved us a lot of time and frustration. It let us focus purely on the logic and design of our project without worrying about hardware issues, like broken wires or dead batteries, right from the start.

3 Overview of a Project

Working of Project

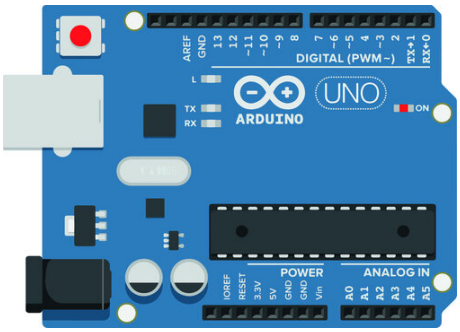
The core idea is pretty simple but effective. We have a "Tilt Sensor" attached to the side of the bottle. This sensor acts as the trigger. Every time you pick up the bottle to take a drink, the sensor tilts, the circuit closes (or opens), and the system registers that you have taken a sip. It then immediately resets its internal timer back to zero, giving you a fresh start.

However, if the bottle sits still on the desk for too long, meaning the user hasn't touched it, the system wakes up and starts a reminder sequence. We designed it to have different "stages" of alerts to escalate the urgency:

- **First Stage (Polite):** A gentle visual reminder using a Green LED. It's subtle and doesn't disturb others around you.
- **Second Stage (Urgent):** A physical vibration and a Yellow LED. This is useful if you are deep in thought and not looking at the bottle.
- **Final Stage (Critical):** A loud buzzer sound and a Red LED. This is the "we are not kidding anymore" stage, used only if you have really ignored the previous warnings.

We designed the progression this way because we didn't want the bottle to just start screaming at you immediately. It tries to be nice first!

4 Arduino UNO



At the heart of our project is the Arduino UNO. You can think of the Arduino as the central brain of the entire operation. It is a small, affordable computer (microcontroller) that reads information from our sensors and decides exactly what to do based on the C++ code we wrote.

Why we chose Arduino UNO

We picked the Arduino UNO because it is incredibly beginner-friendly and perfectly suited for this kind of prototype. It uses the ATmega328P chip, which is powerful enough to handle all our timers, sensor readings, and output signals without any lag. Plus, the Arduino community is huge; if we ever got stuck on a coding problem, we could easily find forums and guides to help us out.

Its Role in Our System

In our specific project, the Arduino is doing a lot of multitasking:

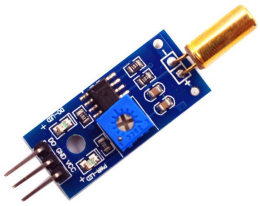
- It constantly "listens" to the digital signal from the tilt sensor to see if the bottle is moving.
- It acts as a stopwatch, keeping track of time (counting milliseconds, seconds, and minutes since the last drink interaction).
- It controls the traffic light LEDs (Green, Yellow, Red) based on how much time has passed.
- It triggers the vibration motor and the buzzer when the urgency level increases.
- It communicates via the I2C protocol to send text messages to the LCD screen, keeping the user informed.

Without the Arduino, our components would just be separate parts doing nothing. The Arduino ties everything together into a cohesive, smart system.

5 Other Hardware Components and Their Purpose

We didn't just pick parts randomly; we carefully selected each component to play a specific role in creating a good user experience. Here is a breakdown of what we used and why:

Tilt Sensor



This is our main input device. Inside, it usually contains a small metal ball that rolls around. It acts like a switch that opens or closes depending on the angle of the bottle. We used this to detect if the bottle is upright (sitting on the table) or tilted (being drunk from). It is a simple, mechanical way to know when the user has "interacted" with the bottle without needing complex accelerometers.

LEDs (Green, Yellow, Red)

We decided to use a traffic light system because it is a universal language, everyone intuitively understands what the colors mean:

- **Green LED:** This turns on first. It's a soft, friendly reminder. It says, "Hey, you're doing okay, but maybe take a sip soon?"
- **Yellow LED:** This means "Caution." You haven't drunk water in a while, so the urgency is going up.
- **Red LED:** This is the danger zone. It means you have waited way too long and is likely dehydrated!

Vibration Motor

We realized during our design phase that sometimes, you might be looking away from the bottle, maybe at your computer screen, so you wouldn't see the LEDs blinking. That's why we added a vibration motor. It buzzes the bottle physically during the Yellow stage. It's great for noisy environments or when you are deeply focused, acting like a silent notification on your phone.

Piezo Buzzer

We added the buzzer for the final, Red stage. If you have ignored the lights and the vibration, the system uses sound to get your attention. We kept this for last because we didn't want the device to be annoying or disruptive in a library or office, but sometimes you need a loud noise to break your hyper-focus.

I2C LCD Display

To make our project look more professional and user-friendly, we added a 16x2 LCD screen. We used an I2C module with it, which is a fancy way of saying we only needed two wires (SDA and SCL) to connect it instead of the usual sixteen wires. This screen talks to the user, displaying messages like "Time to Drink!" or "Great Job!" so you always know exactly what the system is thinking.

6 Pin Configuration

Connecting everything correctly was crucial for the code to work. Here is a list of which pins we used on the Arduino to connect our components. Keeping this organized helped us debug issues later:

Component	Arduino UNO Pin
Tilt Sensor	Digital Pin 2
Green LED	Digital Pin 3
Yellow LED	Digital Pin 4
Vibration Motor	Digital Pin 5
Red LED	Digital Pin 6
Buzzer	Digital Pin 7
LCD SDA	Analog Pin A4
LCD SCL	Analog Pin A5

7 Software Design Methodology

Writing the code was one of the most interesting and challenging parts of this project. We faced a specific problem early on: if we used the standard delay() command (which pauses the Arduino for a set time), the system would "freeze" or sleep. This meant if you picked up the bottle while the light was on, the Arduino wouldn't notice!

So, we decided to use a State Machine approach combined with non-blocking timers (millis()). Instead of pausing the code, our program constantly loops thousands of times a second, asking: "What time is it?" and "What state am I in?". This allows the Arduino to do two things at once: blink an LED and listen for the tilt sensor simultaneously. This makes our system feel very responsive and snappy; as soon as you touch the bottle, the alarm stops instantly.

8 System States

We designed the system to flow through different states logically. Here is how it plays out in a real-life scenario:

The Idle State (Everything is Good)

This is the normal, resting state. You have just drunk water, and the timer is reset. The system is just quietly counting time in the background, waiting for the next interval. The power consumption is lowest here.

The Green Alert State

After a set time passes (like 30 minutes of inactivity), the Green LED turns on. The LCD might display "Hydrate!". It's polite and silent. If you pick up the bottle now, the system says "Thanks!" and resets back to Idle.

The Yellow Alert State

If you ignore the green light for another 15 minutes, things get serious. The Yellow LED turns on and the vibration motor starts buzzing periodically. This is a tactile warning. It's hard to ignore a vibrating bottle on your desk!

The Red Alert State

This is the final level. If you still haven't picked up the bottle, the Red LED flashes rapidly and the buzzer starts beeping. The system is basically shouting at you now. As soon as you tilt the bottle to drink, the noise stops, the lights go out, and the timer restarts the whole cycle.

9 LCD Display Operation

We programmed the LCD to update only when something actually changes (like moving from Green to Yellow state). This prevents the screen from flickering or looking glitchy, which can happen if you try to write to an LCD too fast. It provides clear text feedback so the user isn't just guessing what the blinking lights mean.

10 Limitations

Even though we are proud of our project, we know it has a few limitations that we identified:

- Tilt isn't Drinking: The sensor only knows if you moved or tilted the bottle. It doesn't actually measure if water left the bottle. You could technically "cheat" the system by just tipping it and putting it back down without drinking!
- Fixed Timers: Right now, the time intervals (30 mins, 45 mins, etc.) are hard-coded into the Arduino. You can't change them on the fly without plugging it into a computer and reprogramming it.
- Power: It currently runs on the Arduino's power source (USB), so it's not fully portable yet. It would need a battery pack to be truly mobile.

11 Future Scope

If we were to work on this more in the future, we have some cool ideas to make it even better:

- Flow Measurement: We could add a water flow sensor inside a straw mechanism to measure exactly how many milliliters you drink.
- Wireless Connectivity: We could add a Bluetooth module (HC-05) to connect it to a smartphone app. This would allow you to track your hydration stats over days or weeks.
- User Customization: We could add physical buttons and a menu on the LCD to let the user set their own reminder times (like every 45 mins or 60 mins).

12 Conclusion

Building the Smart Water Bottle Reminder System was a fantastic learning experience for us. We learned how to combine hardware (sensors, motors, lights) and software (logic, state machines) to solve a genuine health problem.

It showed us that embedded systems aren't just about connecting wires and writing code; they are about designing something that interacts with people and helps them live better lives. We successfully created a device that is smart, responsive, and helpful, and we are really happy with how it turned out.